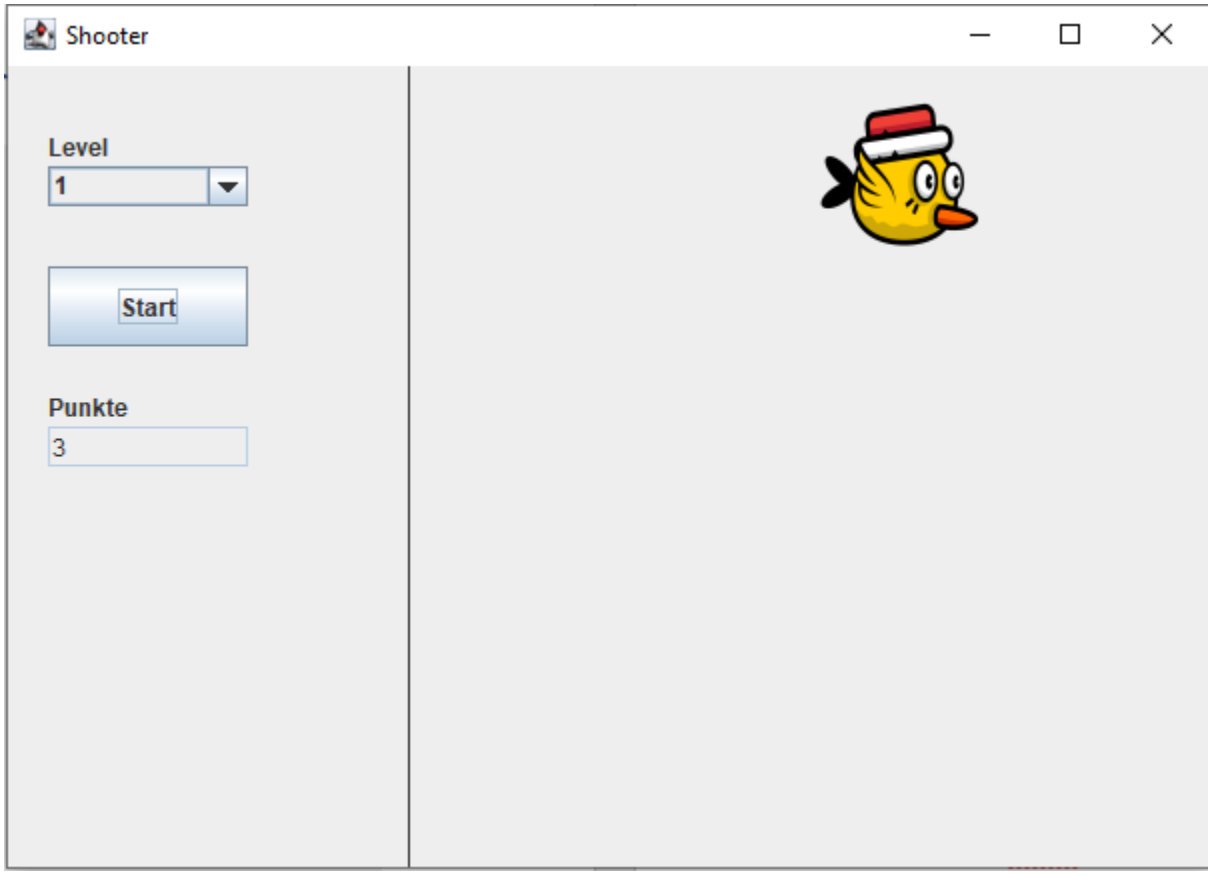
Übung : Shooter – Steuerung mit Events

In dieser Übung wollen wir unser Wissen bezüglich Events und Reaktionen auf Events dazu nutzen, eine kleine Spielapplikation zu programmieren.

Das Panel wird vertikal bei einem Drittel der Breite nach Eingabe und Spielfeld mit einem Strich getrennt. Der Schwierigkeitsgrad wird durch die **JComboBox** zwischen 1 und 3 eingestellt. Mit dem StartButton wird das Spiel gestartet. Dann erscheint Flappy (der Vogel) für eine kurze Zeit zufällig auf dem Spielfeld. Per Mausklick soll er abgeschossen (oder besser gekitzelt) werden. Falls er erwischt wurde, gibt es einen Punkt in dem nicht editierbaren **JTextField**. Bei Änderung des Levels muss erneut Start gedrückt werden. Mit höherem Level wird das Spiel schneller. Die Texte sollen als **JLabel** jeweils oberhalb der entsprechenden Komponenten erscheinen.



Aufgabe 1: Layout init() Methode

Platzieren Sie die Elemente im Eingabefeld in etwa der obigen Skizze. Registrieren Sie alle Listener zu den entsprechenden Komponenten. Die Komponenten sind als Attribute bereits angelegt. Erzeugen Sie die Items der ComboBox wie folgt: `cbLevel.addItem("1");`
Für die weiteren 2 und 3 entsprechend. Mit `cbLevel.getSelectedItem().equals("2")` kann später geprüft werden, welche Auswahl getroffen wurde.

Verschaffen Sie sich einen Überblick über die vorhanden Attribute und Methoden der Klasse.

Aufgabe 2: actionPerformed() Methode

Damit wir möglichst schnell den Flappy sehen und unsere Implementierung testen können, fangen wir an, den Callback des Start Buttons zu beleben. Hier wird die boolsche Variable **pause** auf **false** gesetzt, so dass das Spiel beginnen kann. Dann wird ein **SimpleTimer** mit der levelabhängigen **time** gestartet. Da dies während des Spiels variieren kann, wird bei Vorhandensein eines **timers** dieser zunächst entfernt.

```
pause = false;
if (timer != null) {
      timer.removeTimerListener(this);
}
timer = new SimpleTimer(time, this);
timer.start();
```

Da der **timer** nicht nur einmalig gestartet werden kann, implementiert das ShooterPanel den **SimpleTimerListener** und hat folglich eine **timerAction**()-Methode. Der **timer** wird als Attribut in der Klasse angelegt und ist innerhalb der Klasse überall verfügbar. Deswegen ist die Verwendung etwas anders als in den ersten Übungen, wo wir ihn anonym implementiert haben. Auch wenn Sie den Code von oben kopieren können, vergleichen Sie es mal mit der PseudoCode Beschreibung in der Vorlage.

Aufgabe 3: paintComponent() Methode

Implementieren Sie die Methode gemaess Beschreibung im Code. Flappy soll als Referenzpunkt die Bildmitte haben. Die Breite und Höhe des Bildes ist in der Konstanten **GROESSE** definiert, die Sie verwenden sollen. Verwenden sie die `g.drawImage(imgage, x, y, null);` Methode und setzen Sie das passende Bild und die Koordinaten, so dass x,y der Bildmitte entspricht.

Da die **getXPos()** und **getYPos()** mit einem Defaultwert implementiert sind, sollte Flappy jetzt nach Drücken von Start auf dem Spielfeld erscheinen.

Aufgabe 4: Implementierung getXPos(), getYPos(), itemStateChanged()

- **getPosX**() soll einen zufälligen x-Wert innerhalb des Spielfelds (rechten 2/3 des Panels).
- **getPosY**() soll einen zufälligen y-Wert innerhalb des Spielfeldes zurückgeben.

Da wir ja zur Bildmitte positionieren, darf Flappy leichtüber das Spielfeld hinausragen. Das muss nicht berücksichtigt werden. Nun sollte Flappy zufällig durchs Spielfeld wandern.

-**itemStateChanged**() unterbricht das Spiel und schreibt das Attribut time je nach Auswahl.

Aufgabe 5: Trefferdetektion – mousePressed()

Wenn Flappy mit der Maus angeklickt wird, sollen die **anzahlPunkte** hochgezählt und das Textfeld aktualisiert werden. Überlegen Sie wie der Treffer über die Maus- und Bildkoordinaten detektiert werden könnte.

Hinweis:
Beträge der Differenzen könnten relativ einfach zum Ziel führen.